

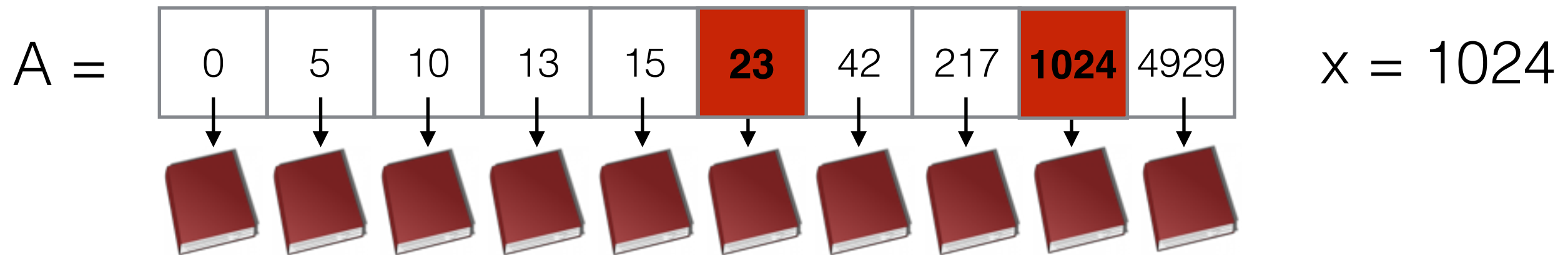
Data Structures in Java

Lecture 2: Interfaces and Generics.
Introduction to Big-O notation.

09/12/2016

Daniel Bauer

Finding an Entry x in a Sorted Array: Binary Search



- Approach 2: **Use the property that the list is sorted.**
 - Find entry y in the middle of A : $y = A[A.length/2]$
 - if $(y == x)$ we found the entry.
 - if $(y < x)$ continue search on second half of A .
 - if $(y > x)$ continue search on first half of A .

Binary Search

```
public static int binarySearch(Integer[] a, Integer x) {  
  
    int low = 0;  
    int high = a.length - 1;  
    int mid;  
  
    while (low <= high) {  
        mid = (low + high) / 2;  
        if (a[mid] < x) { // x is in second half  
            low = mid + 1;  
        } else if (a[mid] > x) { // x is in first half  
            high = mid - 1;  
        } else {  
            return mid;  
        }  
    }  
    return -1;  
}
```

Representing Persons

```
public class Person {  
  
    private String firstName;  
    private String lastName;  
  
    public Person(String last, String first) {  
        lastName = last;  
        firstName = first;  
    }  
  
}
```

The Comparable Interface

```
package java.lang;

public interface Comparable<AnyType> {
    int compareTo(AnyType other);
}
```

`compareTo` returns
negative int if this < o
positive int if this > o
0 if this == o

- comparison usually involves querying some member of `other`.
- The type parameter makes sure that all `other` objects have these fields.

Generic Binary Search

```
public static <AnyType extends Comparable<AnyType>>
    int binarySearch(AnyType[] a, AnyType x) {

    int low = 0;
    int high = a.length - 1;

    int mid;

    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid].compareTo(x) < 0) { // x is in second half
            low = mid + 1;
        } else if (a[mid].compareTo(x) > 0) { // x is in first half
            high = mid - 1;
        } else {
            return mid;
        }
    }
    return -1;
}
```

A Comparable Person Class

```
public class Person implements Comparable<Person> {  
  
    private String firstName;  
    private String lastName;  
  
    public Person(String last, String first) {  
        lastName = last;  
        firstName = first;  
    }  
  
    public int compareTo(Person other) {  
        int lastNameComp = lastName.compareTo(other.lastName);  
        if (lastNameComp == 0)  
            return firstName.compareTo(other.firstName);  
        else  
            return lastNameComp;  
    }  
}
```


Algorithms

- An algorithm is a clearly specified set of simple instructions to be followed to solve a problem.
- Algorithm Analysis — Questions:
 - Does the algorithm terminate?
 - Does the algorithm solve the problem? (correctness)
 - What resources does the algorithm use?
 - Time / Space

Analyzing Runtime

- We usually want to compare several algorithms.
 - Compare between different algorithms how the runtime $T(N)$ grows with increasing input sizes N .
- We are using Java, but the same algorithms could be implemented in any language on any machine.
- How many basic operations/*steps* does the algorithm take? All operations assumed to have the same time.

Worst and Average case

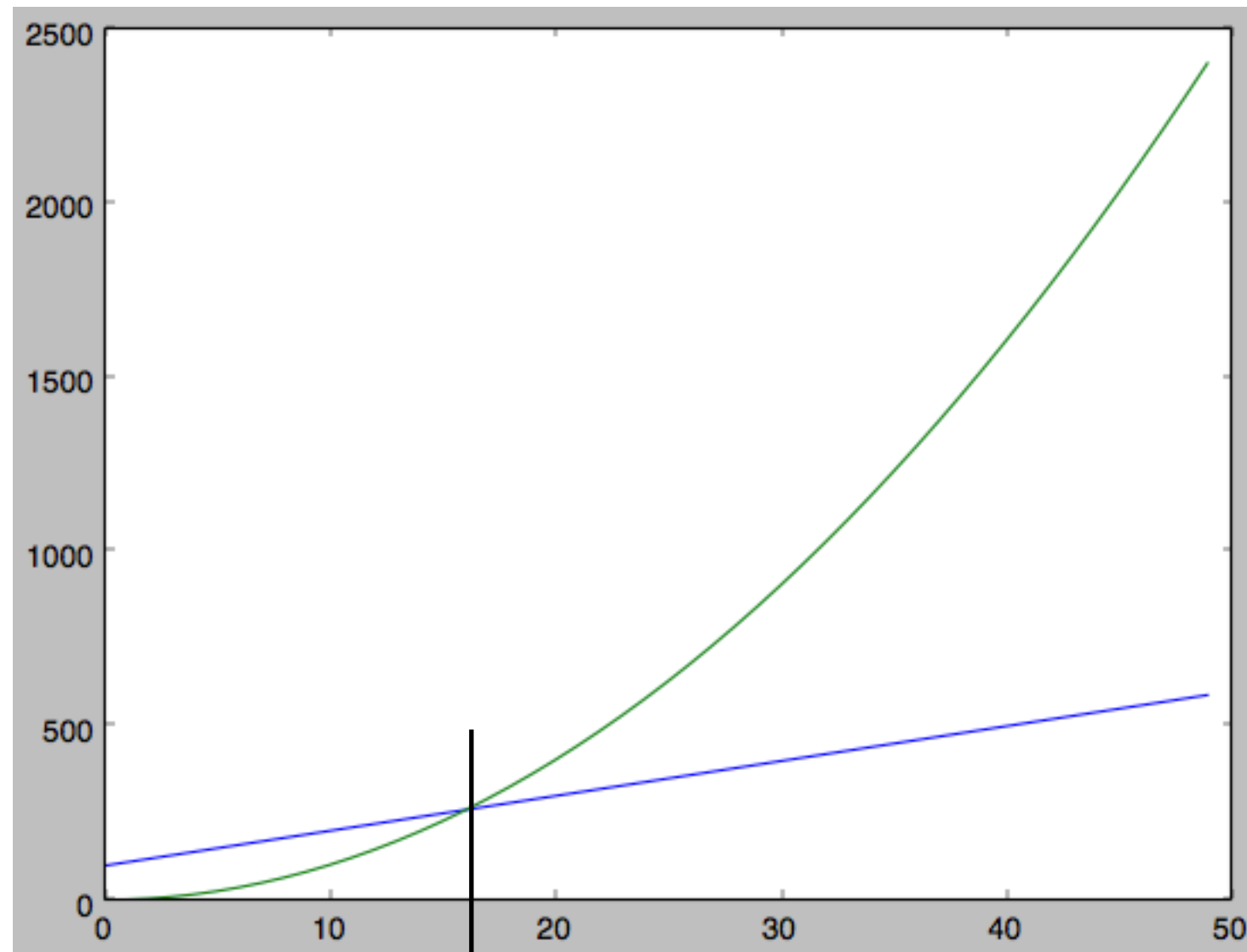
- Usually the runtime depends on the type of input (e.g. sorting is easy if the input is already sorted).
- $T_{worst}(N)$: *worst case* runtime for the algorithm on ANY input. The algorithm is **at least** this fast.
- $T_{average}(N)$: *Average case analysis* — expected runtime on typical input.
- $T_{best}(N)$: Rarely we are interested in the *best case analysis*.

Comparing Function Growth: Big-O Notation

$T(N) = O(f(N))$ if there are positive constants c and n_0 such that $T(N) \leq cf(N)$ when $N \geq n_0$.

“ $T(N)$ is in the order of $f(N)$ ”

“ $f(N)$ is an upper bound on $T(N)$ ”



$$f(N) = N^2 + 2$$

$$T(N) = 10N + 100$$

e.g. $c = 1$, $n_0 = 16.1$

Typical Growth Rates

